Ermeline Bresson
12411807

INFO2
Enseignant : M. Youcef

TP2 Base de données avancées

Exercice 1 :

Question 1 : Nom du département au budget le plus élevé

```
select dept_name
from department
where budget in (select max(budget) from department);
```

Results | Explain | Describe | Saved SQL | History

DEPT_NAME
Finance

Question 2 : Salaires et noms des enseignants qui gagnent plus que le salaire moyen

```
select name, salary
from teacher
where salary > (select avg(salary) as moyenne from teacher);
```

Results | Explain | Describe | Saved SQL | History

NAME	SALARY
Wu	90000
Einstein	95000
Gold	87000
Katz	75000
Singh	80000

Question 3 : Pour chaque enseignant, affichage de chaque étudiant ayant suivi plus de deux cours dispensés par l'enseignant ainsi que le nombre total de cours suivis par l'étudiant en utilisant la clause HAVING

```
select teacher.name, student.name, count(*)
from teacher, student, takes, teaches
where teacher.ID = teaches.ID and student.ID = takes.ID and (teaches.course_id = takes.course_id and teaches.sec_id = takes.sec_id
and teaches.semester = takes.semester and teaches.year = takes.year)
group by teacher.name, student.name having count(*) >= 2;
```

Results | Explain | Describe | Saved SQL | History

NAME	NAME	COUNT(*)
Srinivasan	Zhang	2
Srinivasan	Shankar	3
Katz	Levy	2
Srinivasan	Bourikas	2
Crick	Tanaka	2

Question 4 : Pour chaque enseignant, affichage de chaque étudiant ayant suivi plus de deux cours dispensés par l'enseignant ainsi que le nombre total de cours suivis par l'étudiant sans utiliser la clause HAVING

```sql
select t_name, s_name, nombre_cours
from ( select teacher.name as t_name, student.name as s_name, count(*) as nombre_cours
       from teacher, student, takes, teaches
       where teacher.ID = teaches.ID and student.ID = takes.ID and teaches.course_id = takes.course_id and teaches.sec_id = takes.sec_id
       and teaches.semester = takes.semester and teaches.year = takes.year
       group by teacher.name, student.name) resultat
where nombre_cours >= 2;
```

Results | Explain | Describe | Saved SQL | History

T_NAME	S_NAME	NOMBRE_COURS
Srinivasan	Zhang	2
Srinivasan	Shankar	3
Katz	Levy	2

Question 5 : Noms et identifiants des étudiants n'ayant pas suivi de cours avant 2010

```sql
select student.ID, student.name
from student
where not exists (
                select 1
                from takes
                where student.ID = takes.ID and takes.year < 2010)
```

Results | Explain | Describe | Saved SQL | History

ID	NAME
19991	Brandt
23121	Chavez
55739	Sanchez
70557	Snow

Question 6 : Noms des enseignants qui commencent par un E

```sql
select name
from teacher
where name like 'E%';
```

Results | Explain | Describe | Saved SQL | History

NAME
Einstein
El Said

Question 7 : Salaires et noms des enseignants qui perçoivent le quatrième salaire le plus élevé

```sql
select name
from teacher T1
where 3 = (select count(distinct T2.salary) from teacher T2 where T2.salary > T1.salary);
```

Results | Explain | Describe | Saved SQL | History

NAME
Gold

Question 8 : Noms et salaires des trois enseignants qui perçoivent les salaires les moins élevés, par ordre décroissant

```sql
select T1.name, T1.salary
from teacher T1
where 2 >= (select count(distinct T2.salary) from teacher T2 where T2.salary < T1.salary)
order by T1.salary asc;
```

Results | Explain | Describe | Saved SQL | History

NAME	SALARY
Mozart	40000
El Said	60000
Califieri	62000

Question 9 : Nom des étudiants ayant suivi un cours en automne 2009 en utilisant la clause IN

```sql
select distinct student.name
from student, takes
where student.ID = takes.ID and (semester, year) in (('Fall', 2009));
```

Results | Explain | Describe | Saved SQL | History

NAME
Zhang
Shankar
Peltier
Levy

Question 10 : Nom des étudiants ayant suivi un cours en automne 2009 en utilisant la clause SOME

```sql
select distinct student.name
from student, takes
where student.ID = takes.ID and (semester, year) = some (('Fall', 2009));
```

Results | Explain | Describe | Saved SQL | History

NAME
Zhang
Shankar
Peltier

Question 11 : Nom des étudiants ayant suivi un cours en automne 2009 en utilisant NATURAL INNER JOIN

```sql
select distinct student.name
from student
natural inner join takes
where takes.semester = 'Fall' and takes.year = 2009;
```

Results | Explain | Describe | Saved SQL | History

NAME
Zhang
Shankar
Peltier
Levy

Question 12 : Nom des étudiants ayant suivi un cours en automne 2009 en utilisant la clause EXISTS

```sql
select distinct student.name
from student
where exists (
    select 1
    from takes
    where student.ID = takes.ID and takes.semester = 'Fall' and takes.year = 2009)
```

Results | Explain | Describe | Saved SQL | History

NAME
Zhang
Shankar
Peltier
Levy

Question 13 : Paires d'étudiants ayant suivi au moins un cours ensemble

```sql
select s1.name, s2.name
from (select * from student natural inner join takes) s1,
     (select * from student natural inner join takes) s2
where s1.course_id = s2.course_id and s1.sec_id = s2.sec_id and s1.semester = s2.semester and s1.year = s2.year and s1.ID <> s2.ID
and s1.name < s2.name
group by s1.name, s2.name
having count(*) >= 1;
```

Results | Explain | Describe | Saved SQL | History

NAME	NAME
Shankar	Williams
Shankar	Zhang
Levy	Shankar

Question 14 : Affiche pour chaque enseignant qui a effectivement assuré un cours le nombre d'étudiants ayant suivi ses cours. Trié par ordre décroissant

```sql
select teacher.name, count(*)
from (takes inner join teaches using (course_id, sec_id, semester, year)) inner join teacher on teaches.id = teacher.id
group by teacher.name, teacher.id
order by count(*) desc;
```

Results | Explain | Describe | Saved SQL | History

NAME	COUNT(*)
Srinivasan	10
Brandt	3
Crick	2

Question 15 : Affiche pour chaque enseignant même s'il n'a pas réellement assuré un cours le nombre d'étudiants ayant suivi ses cours. Trié par ordre décroissant

```sql
select teacher.name, count(course_id)
from (takes inner join teaches using (course_id, sec_id, semester, year)) right outer join teacher on teaches.id = teacher.id
group by teacher.name, teacher.id
order by count(course_id) desc;
```

Results | Explain | Describe | Saved SQL | History

NAME	COUNT(COURSE_ID)
Srinivasan	10
Brandt	3
Katz	2

Question 16 : Pour chaque enseignant, afficher le nombre total de grades 1 qu'il a attribué

```sql
select teacher.name, count(*) nb_grades_A
from teacher, teaches, takes
where teacher.ID = teaches.ID and teaches.course_id = takes.course_id and teaches.sec_id = takes.sec_id and teaches.semester = takes.semester
and teaches.year = takes.year and takes.grade = 'A'
group by teacher.ID, teacher.name;
```

Results | Explain | Describe | Saved SQL | History

NAME	NB_GRADES_A
Srinivasan	4
Crick	1
Brandt	2

Question 17 : Paires enseignant-élève où un élève a suivi le cours de cet enseignant ainsi que le nombre de fois où l'élève a pu suivre un cours de l'enseignant

```sql
select teacher.name, student.name, count(*)
from (teacher natural join teaches) inner join (takes natural join student) using (course_id, sec_id, semester, year)
group by teacher.name, student.name;
```

Results | Explain | Describe | Saved SQL | History

NAME	NAME	COUNT(*)
Srinivasan	Zhang	2
Srinivasan	Shankar	3
Brandt	Shankar	1
El Said	Brandt	1
Wu	Chavez	1
Einstein	Peltier	1

Question 18 : Paires enseignant-élève où un élève a suivi au moins deux cours dispensés par l'enseignant en question

```sql
select teacher.name enseignant, student.name eleve
from teacher, teaches, takes, student
where teacher.ID = teaches.ID and teaches.course_id = takes.course_id and teaches.sec_id = takes.sec_id and teaches.semester = takes.semester
and teaches.year = takes.year and takes.ID = student.ID
group by teacher.ID, teacher.name, student.ID, student.name
having count(*) >= 2;
```

Results | Explain | Describe | Saved SQL | History

ENSEIGNANT	ELEVE
Srinivasan	Zhang
Srinivasan	Shankar
Katz	Levy
Srinivasan	Bourikas

Exercice 2 :

1. R(A,B,C) et F = { A -> B; B -> C }
    - Clé primaire : A
    - Première forme normale : Oui car il n'y a aucun attribut multivalué
    - Deuxième forme normale : Oui car il n'y a aucune dépendance fonctionnelle
    - Troisième forme normale : Non car on obtient C par transitivité
      -> On décompose en deux relations :
        - R1(A, B)
        - R2(B, C)

2. R(A,B,C) et F = { A -> C; A -> B}
    - Clé primaire : A
      -> La relation est déjà en BCNF

3. R(A,B,C) et F = { A; B -> C; C -> B }
    - Clé primaire : A,B ou A,C
    - La relation est bien en 1FN, 2FN et 3FN mais pas en BCNF
    - C -> B viole BCNF car C n'est pas une superclé
      -> On décompose selon C -> B pour obtenir une BCNF :
      R1(C,B)
      R2(A,C)

Exercice 3 :

Question 1 : F = { A → B, C; C, D → E; B → D; E → A }
En utilisant le système d'Armstrong, on a :

A -> D	C, D -> A	A, B -> C
A -> E	C, D -> B	A, B -> D
B -> E	C, D -> C	A, B -> E
B -> A	C, D -> D	B, D -> A
B -> C	E -> B	B, D -> E

Question 2 :

(a) Calcul de la fermeture de B et de { A, B } :

{B}+ = { A, B, C, D, E }

{ A, B }+ = { A, B, C, D, E }

(b) Montrer que A, F est une superclé

{A}+ = {A, B, C, D, E}
{F}+ = {F}
L'union de ces deux fermetures donne la relation R donc {A,F} est bien une superclé.

(c) {A,F} est-elle en BCNF ?
Non, elle n'est pas en BCNF.
On la décompose donc de la façon suivante :
R1(A,B,C,D)
R2(E,A)
R3(E,F)

Question 3 : Relation R(A,B,C,D,E)

(a) R1(A,B,C) et R2(A,D,E) :
L'intersection de ces deux relations donne A or :
Si A -> B, C dans R1 et A -> D, E dans R2 alors A est une clé dans les deux relations donc la décomposition est sans perte d'information

(b) R1(A,B,C) R2(C,D,E) et F = {A→B,C;C,D→E;B →D;E →A}
L'intersection de ces deux relations donne C or :
{C}+ = {C}
Donc C n'est pas une clé donc on a une perte d'information.